The neural-network bot for Troll Farm

Where the project stands on 2026-09-01

Prepared for the owner by the coordinator (`local_claude_1`)

2026-09-02, 06:3xZ (seventh edition: the campaign ledger — training now improves the bot, Figure 10; the sixth was 09-01 17:2xZ) — everything below is on `main` unless a path says otherwise

In one page

The goal. A bot for the CodinGame game *Troll Farm* whose every command comes from a small neural network — the way the #1 player on the ladder (*delineate*, rating 30.9) plays. Our best hand-written bot ("the champion of record") reads about 18–21 on the same ladder.

The target you set (2026-08-29). A network candidate that beats the champion of record *and* the "orchard 6" bot on our own bench — at least 60 % wins over 400 games against each, three checks in a row — exported as one Rust file under 100,000 characters at ≤ 15 ms a turn, verified, ready for the ladder, **by 2026-10-17**. Milestone for your read: the clone playing whole games against the champion's file, aimed at 2026-09-14.

Where we are, two days later.

- The training environment (a full copy of the game that plays 128 games at once) is built, tested and reproduced by a second agent.
- Every recorded game of the top four players has been turned into a training set: **817,811 decisions from 748 games**, 14 MB.
- A first network, **the clone**, learned by copying those decisions. It plays complete, legal games and **beat the champion's own file in 9 of 48 games** (134 points to 186 on average) — *the milestone, reached two weeks early.* Its games are on file for you (Section 8).
- **Self-play training** started at 04:45Z today. Four runs before noon taught two lessons: the audits' fixes to the trainer, and *training against our weak practice bots made the network worse against the champion.* Since 09:42Z the runs train against **an exact in-process copy of the champion**, built and reproduced within a day.
- **The day's central finding, now proven in five runs: ordinary (full-parameter) self-play from the clone always follows one shape** — a dip by update 500, a partial recovery at 1,000, a collapse by 1,500 — whatever it trains against (weak bots, the exact champion, or the champion alone), whatever the discount, with or without the value-estimate warm-up (Figure 1). Reading the games shows the same signature every time: the multi-step fruit work (harvest, carry home, plant) decays first while the immediate chopping survives (Figure 2).
- **The evening's control experiment explained the "rising" training numbers**: a run's own win rate climbs while its real play falls, because training plays *sampled* moves against a pool that is 60 % weak opponents. Measured on the clone itself: sampled commands lose six wins and thirty points (9 of 48 becomes 3–4). So the bench's argmax play — the way the shipped bot actually decides — is the only honest measure, and it is now the only one used.
- **The staged recipe — the winner's own stage 4 — is the first run that neither erodes nor collapses.** The audits pointed at the shared body of the network (the value estimate's learning pushes the movement head through it), so codex_1 rebuilt the trainer within an hour: the movement network frozen bit for bit and executed exactly as we ship it, only the purchase head and the value estimate learn. That run (I) read **9 10 9** against the champion's file —

at the clone's bar, once above it — then drifted to 6 and 5 as its leash to the clone faded, and stopped by its pre-stated rule. The lesson is sharp: the freeze stops the catastrophe; the leash must not fade. The best artefacts so far: the clone (9 of 48) and run I's update-1,000 snapshot (10 of 48).

- **The cluster night died and was rebuilt.** The first six 12-hour jobs were killed by their own wall-clock limits at 01:29Z — my error: I gave them a step budget needing ∼16 hours inside a ∼13-hour limit — and their checkpoints lived only inside the jobs, so nothing was retrievable; the four not yet dead were aborted to save your cluster grant. Two repairs are on `main`: every job now uploads a salvage copy of its newest checkpoint every half hour, and the launcher carries the staged-scope flag. **Three jobs run now under consistent 17-hour limits**: the long full-parameter answer (a2), the corrected-objective variant (e2), and *the staged scope with a leash that never fades* (i2) — exactly the lever run I's drift named. Results between 19:00Z and 20:00Z today.
- **Phase 4 is complete: the clone exists as one Rust file** (`cgauto/submissions/candidate-nn-clone.rs`), now with a runtime processor check and a proven fallback — an audit found the first version executed AVX2 instructions unconditionally, which on a platform machine without them would crash before the first command; the repair was delivered, and the shipping binary's fallback path was proven AVX-free by disassembly. 54,218 characters (83,282 in the platform's likely counting units, of 100,000); 6.5 ms a turn; move-for-move identical to the Python clone on both processor paths; reproduced twice. **Not submitted**, and not called ladder-ready until the three-run timing certificate on this computer (taken when a real candidate ships) and your word.

Added 2026-09-01 (Section 3). The three overnight jobs answered as feared (2, 3 and 4 of 48 — depth hurts, full-parameter or staged). The reviewer's prime suspect, the *entropy bonus*, was put to a frozen test: two arms from the same clone differing in that one number, trained to 2,709 updates on the cluster, benched on a locked 144-cell panel at two ages — **verdict `NOT CONFIRMED`: removing the bonus changes nothing** (24 / 21 wins against 23 / 22 of 144; effect 0.000, interval $[-0.017, +0.021]$), replicated on this computer. What the day did find is *why* the runs decay: the planner learns from a signal that is **97.7 % the value estimate's own opinion and 2.3 % observed outcome**, because wood's whole value is paid on the last turn while the trainer looks ahead about nine turns; under that reward only the 0.13 % of rows that are game endings carry any reward at all. The first lever against it — paying half of wood's value on delivery — is training on the cluster as this edition is written (Figures 8–11).

What was next on 08-31. The three cluster jobs return by ∼20:00Z and their final snapshots are benched here against the champion's file; the clone's 9 of 48 stays the bar. The gradient instrument (written by claude_1, one correction pending) then says *why* the value estimate's learning bends the policy, and the next lever is picked on that evidence: the never-fading leash (already running as i2), the winner's final stage (a joint fine-tune at a tiny rate from run I's update-1,000 snapshot), or separating the value estimate's body from the policy's. A snapshot above 55 % starts the card's 400-game checks against the champion and orchard 6; then the export (a one-command step), the bed, the timing certificate, and — on your word, when the platform is free — one hour on the ladder.

1 The plan and its status

The work is one card, `coordination/tasks/20260829-nn-bot-way-b.md`, in five phases. Each phase has a "done" condition, a "dead" condition and a budget, like every card on the board.

	what	budget	status
0	the software runtime on this computer; the old (July) pipeline re-run end to end	1–2 days	**done** 08-29 14:2xZ
1	the full-game training environment (Rust), with a Python wrapper	6 days	**done** 08-29 21:1xZ, in one day; reproduced
2	the training set, the bench (our judge), the clone	7 days	**tools done and reproduced** 08-30 03:2xZ; **the clone's games on file** 04:5xZ — the milestone
3	self-play training from the clone, with checks every few hours	2–4 weeks	**running, no gain yet**: seven host runs in two days mapped the failure (full-parameter always collapses; the staged recipe holds the bar, then drifts as its leash fades); best snapshots 9–10 of 48 = the clone's level; three rebuilt cluster jobs (results ∼20:00Z) and the gradient instrument decide the next lever
4	export to one Rust file, the timing and size checks, one ladder hour	3 days + 1 hour	**complete** 08-30, in one day, plus the portability amendment the same evening (a runtime processor check with a proven fallback; both paths move-for-move identical; reproduced twice); open: the three-run timing certificate on this computer, taken when a real candidate ships; the ladder hour waits for a candidate that beats the clone, and for the platform

2 What was done, step by step

2026-08-29 — the analysis and your decisions

I read what the repository already knew: the #1 player's own description of his network (we hold his text), our July attempts at learning (they stopped for reasons that are now avoided), the game engine's speed and accuracy, the recorded games. Conclusion: every piece existed except the assembly. You decided: open the line; way B — *copy the top players first, then improve by self-play*; you judge the clone's games yourself.

2026-08-29 — the environment (Phase 1)

codex_1 built the environment on the VM in a day. Its written interface (what the network sees, how commands are encoded, how a turn is split into decisions) was audited the same day by *chatgpt_1*, on your instruction, and **ten real defects** were found and fixed before any number was accepted — among them: the starting troll was created with the wrong strength on both sides of the test, so the test could not have caught it; a counter of illegal commands that could never be non-zero; a check of *whether* a game was replayed right but not *when* it ended; and a vocabulary of buyable trolls copied from delineate that the other top players do not respect (see Section 5). The final version passed its check — 1,000 self-play games replayed through an independent copy of the rules with no disagreement — and *claude_1* reproduced it byte for byte.

2026-08-29/30 — the training set, the bench, the clone (Phase 2)

claude_1 built the three tools; *codex_1* reproduced them; I ran the training set on this computer (1 minute 42 seconds) and trained the clone (4 passes over the data, 2 hours). The bench — the clone

against the champion's actual submitted file, on 24 real ladder maps, once on each side — gave the milestone numbers. Figures 3–5.

2026-08-30 — self-play (Phase 3), and what the first day of it taught

Self-play works like this: the network plays 128 games at once on real ladder maps, is nudged toward what wins, and a leash ("the anchor") keeps it close to the clone at first. Seven runs were started today on this computer; none has yet produced a snapshot better than the clone, and the afternoon went into finding out why.

- **04:45Z, run A** — against six of our hand-written bots and frozen copies of itself. Its win rate against that mix rose from 0 to 42 % in three hours. *But its snapshot lost to the champion's file worse than the clone it started from*: 2 wins of 48 against the clone's 9, 87 points against 134. Meanwhile the audits found two defects in the trainer: the reward of a turn was still being discounted once per troll inside the turn, and the "which troll am I saving for" planes shifted the network's plan decision at the hand-over from copying to self-play. Both fixed.
- **07:40Z, run B** and **08:39Z, run C** — restarted from the clone with the fixes (run C once a third such plane was found). Run C's snapshot, still trained against the old practice mix, again lost to the champion worse than the clone: 3 wins of 48. The lesson was now certain: *what the network trains against matters more than how fast it trains*; our practice bots teach habits that do not transfer.
- **09:42Z, run D — the longest run.** codex_1 was chartered at 07:45Z to build an exact in-process copy of the champion as a training opponent; it delivered at 09:12Z — the copy matches the champion's submitted file on 200 of 200 games and every one of their 49,945 turns — and claude_1 reproduced the proof by 10:02Z. Run D trained against that copy (weighted highest in its pool) from the clone, with all fixes, for 1,138 updates (4.7 million decisions). **Its snapshots against the champion's file: 3 wins of 48 after 500 updates, 4 of 48 after 1,000** (the clone: 9); 134 → 108 → 82 points. Its practice win rate stayed flat near 26 %. Stopped at 14:2xZ with its trend established.
- **13:5xZ–14:2xZ, what the games say.** The first guess — that the network learned to chop everything for the per-wood bonus — was *refuted by counting*: per game the clone chops 94 times, harvests 38, plants 25; the update-500 snapshot chops 70, harvests 27, plants 19; update 1,000 chops 71, harvests 17, plants 14 (Figure 2). The snapshots do less of everything productive and buy fewer trolls (44 games of 48 with a purchase for the clone, 33, then 26); the champion fells the trees while they idle, so 29 of 48 games ended early with no trees left. Two causes fit. (1) *The objective*: the trainer discounts the reward by 0.3 % per turn, so from turn 50 the final score is worth a quarter of its face value and the small per-wood bonus paid every turn dominates. (2) *The untrained value estimate*: the network has a second output that guesses how the game will end, used to judge every move; the clone never trained it, so the first hundreds of updates push the policy with noise while the exploration bonus loosens the leash.
- **13:5xZ, run E and 14:2xZ, run F — one run per cause.** Run E: the discount reduced to 0.1 % per turn, no per-wood bonus, the real end score, the leash never below 0.05. Run F: the same, plus a *warm-up* added to the trainer today — for the first 300 updates only the value estimate trains while the policy stays frozen bit for bit, then the normal loop with the policy's learning rate cut to 0.3. Both were **stopped from outside the session at 14:41Z**, one update short of their first snapshot, while the machine was busy with your work: together they used 20 threads, above the 14 the target allows. Since 14:4xZ **one run on this computer**: run F again (`ppo-f2`), 8 threads at the lowest priority. Run E moved to the cluster.
- **15:5xZ, the first test of the remedies.** `ppo-f2`'s update-500 snapshot — 200 policy updates after its 300-update warm-up, at a third of the learning rate, under the corrected objective — against the champion's file: **5 wins of 48** (3 on seat 0, 2 on seat 1), 123.5 points to 177.8; 19 games ended early (run D's update-1,000 snapshot: 31), no loops. Per game it chops 75 times, harvests 37, plants 21 and buys a troll as often as the clone (the clone: 94, 38, 25) — the harvesting and the purchases held, the chopping fell. Between the clone and run D at the same age; the

erosion per update is about run D's, on one 48-game bench. Not a gain. The update-1,000 snapshot, about 16:25Z, decides whether the remedies hold the line or only slow the drift.

2026-08-30 — the network as one file (Phase 4's engineering), delivered

codex_1 was chartered at 11:1xZ and delivered at 14:57Z, after one amendment of mine (the shipped bot must know which side it plays: the game's text protocol carries no such field and the map's shape does not tell — the bot reads it once, on turn one, from its starting troll's number, and refuses to answer if the numbers are not what the rules promise). Three programs: the *exporter* turns a checkpoint into integer weights (72,660 bytes; effective 16-bit precision, so the file plays exactly as the floating-point network); the *generator* writes the one Rust file — it copies the game, the mask, the command codec and the 104-plane builder from the training environment's own source rather than re-writing them, and pins their hashes; the *bed* compiles the file, plays the 24 bench maps on both seats and compares every command with the Python clone on the same games, checks the observation and mask bytes directly on both seats, times every turn and counts the characters. codex_1's numbers and mine, from a clean checkout of its commit on this computer, agree: 48 of 48 games and 13,206 of 13,206 commands identical; 52,854 characters; first turn 14.8 ms at most, ordinary turns 6.5 ms in the middle and 10.6 ms at the 99th percentile (the limits: 500 and 15). The 370 recorded games with a seat-0 first turn all obey the troll-number rule. Merged onto `main`; claude_1 reproduced it on the VM the same hour. **Nothing is submitted.**

The evening added the portability amendment. chatgpt_1's audit found the generated file executed AVX2 processor instructions unconditionally — on a platform machine without them it would crash before printing its first command, and no test on our machines could see it, because both have them. codex_1 delivered the repair within the hour: the bot checks the processor once at start and falls back to a plainer kernel with the identical arithmetic order, and the bed now compiles and plays *both* paths — 48 of 48 and 13,206 of 13,206 each. claude_1 reproduced that too, and added the proof no bed can give: a disassembly of the shipping binary showing its fallback kernel contains not one AVX instruction. The size gate now counts UTF-16 units, the platform's likely measure (83,282 of 100,000). Two rules were frozen alongside: the timing gate is three quiet runs on this computer (median 99th-percentile ≤ 15 ms, every run ≤ 20), taken when a real candidate ships; and until that certificate and your word, no file of this line is called ladder-ready.

2026-08-30, evening — the shape proven, the reasons narrowed

The remedies' run (F2) recovered to 7 of 48 at update 1,000 and collapsed to 2 at 1,500. A champion-only run (G) eroded the same way (5 4). A run with the value target undiscounted (H) read 3 8 2 — and an audit showed my "the discount is now swept" conclusion was wrong: in our estimator the per-turn credit trace is $\gamma \cdot \lambda$ with $\lambda = 0.95$, so that run changed the policy's credit by 0.1 %; only the value estimate's target changed (and got harder to fit). Five runs, one shape (Figure 1).

Two control measurements sharpened the picture. *The decoding factorial*: the clone benched under every combination of decodings — plan and commands each by most-likely-move or by draw — reads 9 / 8 / 3 / 4 of 48: the *command* decoding carries the whole gap. Training plays drawn commands, the shipped bot plays most-likely — so a run's own "win rate" (drawn play, 60 % weak opponents) can rise while the real play falls, which is exactly what happened. And *the audits* (chatgpt_1, five that evening) named an unmeasured mechanism common to every collapsing run: the value estimate's learning flows back through the network's shared body and moves the movement head, hardest when the value target is hardest. claude_1 wrote the measuring instrument the same evening (one correction of its reviewer pending); its verdict comes with the morning.

2026-08-30, night — the staged recipe holds the bar

The winner's recorded stage 4 — freeze the movement network, train the plan selector and the value head on pure end score — became a trainer mode within the hour (codex_1): the movement plays the frozen policy's *most likely* move, exactly as the shipped bot would, and every learning signal except the value estimate's is computed over plan decisions only. Run I, under that mode: **9 10 9** against the champion's file — at the clone's bar for 1,500 updates, once one win above it, the first run of the programme that neither eroded nor collapsed — then 6 and 5 as its leash to the clone faded toward 0.05, and it stopped by its pre-stated two-reads rule. Its practice numbers, for the first time, tracked the bench (drawn plans over a frozen argmax executor against the champion alone). The conclusion written on the card: *the freeze stops the catastrophe; the leash must not fade* — and a cluster job with a never-fading leash was launched that hour.

The cluster night, died and rebuilt. You said one run of 3.5 days is a job for the cluster; six 12-hour jobs went up at midday. At 01:29Z the first two were killed by their operations' wall-clock limits: I had given every job a 60-million-decision budget needing ∼16 hours inside a ∼13-hour limit — two numbers I set inconsistently — and the checkpoints lived only inside the jobs, so thirteen hours of training were unretrievable. The four not yet dead were doomed identically and were aborted to save your grant. Both repairs are on `main`: every job now uploads a salvage copy of its newest checkpoint and log every half hour beside its final output, and the launcher passes the staged-scope flag. **Three jobs run now**, 60 million decisions each under consistent 17-hour limits, all from the clone: *a2*, the full-parameter run-of-record recipe — the long-horizon answer (the dead first attempt's telemetry reached a practice win rate of 29 % at 48 million decisions, still climbing — tantalizing, but its pool makes that number soft, and its snapshot is gone); *e2*, the corrected objective; *i2*, **the staged scope with the leash pinned at 0.1 for the whole run**. Their final snapshots return between 19:00Z and 20:00Z today and are benched here.

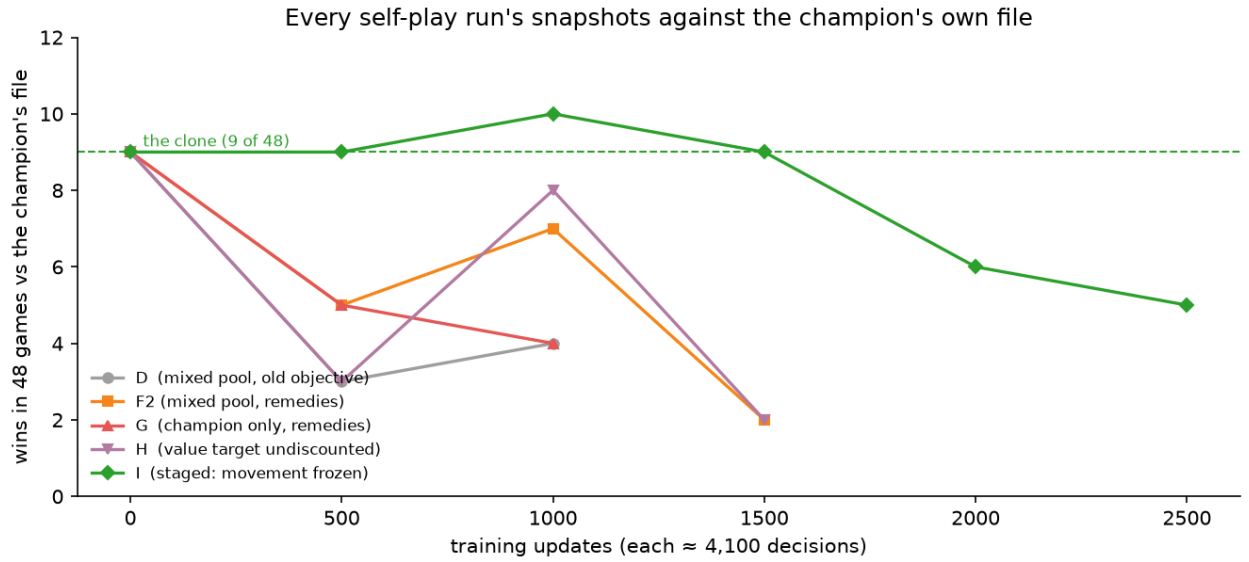

Figure 1: The programme's central picture. Each line: one self-play run's snapshots, benched in 48 games against the champion's own file (every run starts from the clone, dashed at its 9 of 48). The four full-parameter runs — whatever the pool, the discount or the warm-up — dip, partly recover, and collapse by update 1,500. The staged run (I, green): the movement network frozen, only the purchase head and the value estimate learn — holds the clone's bar for 1,500 updates and once tops it, then drifts as its leash to the clone fades. The overnight cluster job i2 repeats it with a leash that never fades.

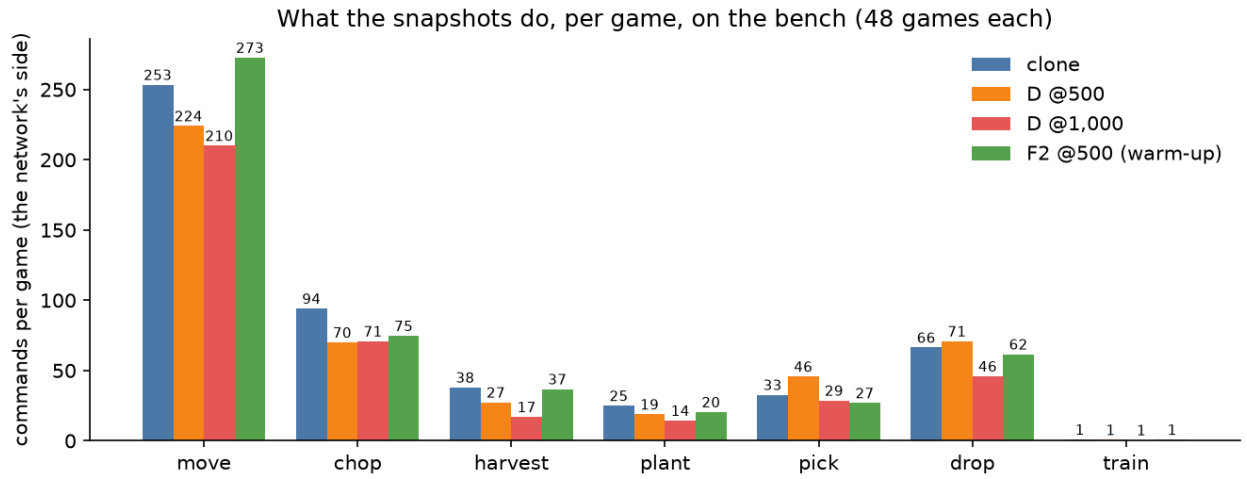

Figure 2: Why the score falls: what the network's trolls do per game on the bench — the clone, run D's two snapshots, and the first snapshot of the run with both remedies (F2). Run D's snapshots chop *less*, not more — the "chops everything" guess is wrong; they harvest, plant and buy less too. Self-play so far teaches inaction, which is what the two remedies (a truer objective; a warmed-up value estimate) are meant to cure; F2's harvests, plantings and purchases stayed near the clone's.

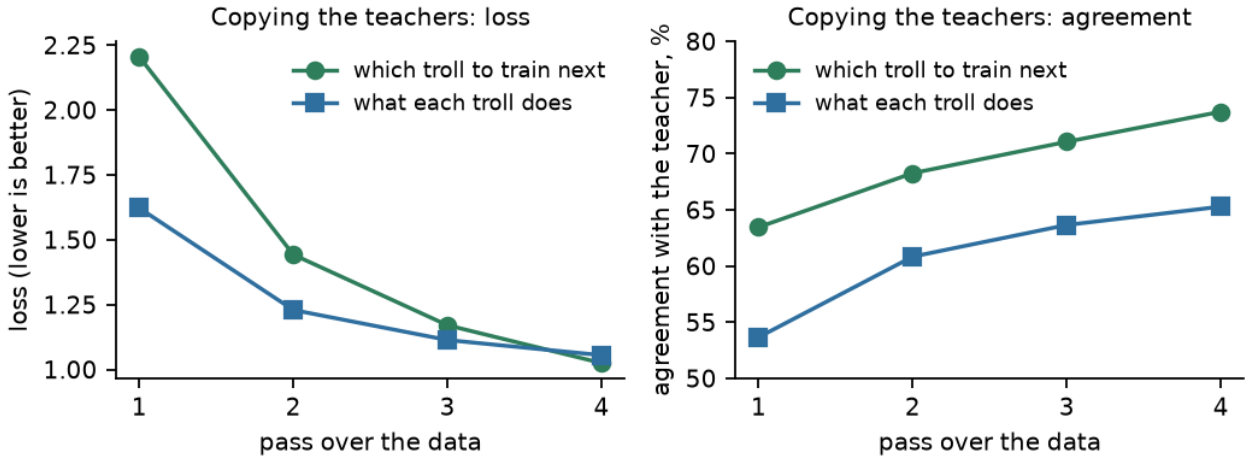

Figure 3: The clone learning to copy the teachers. Left: the loss (an error measure) of its two decisions — which troll to save for, and what each troll does — over the four passes. Right: how often it agrees with the teacher on the training moves.

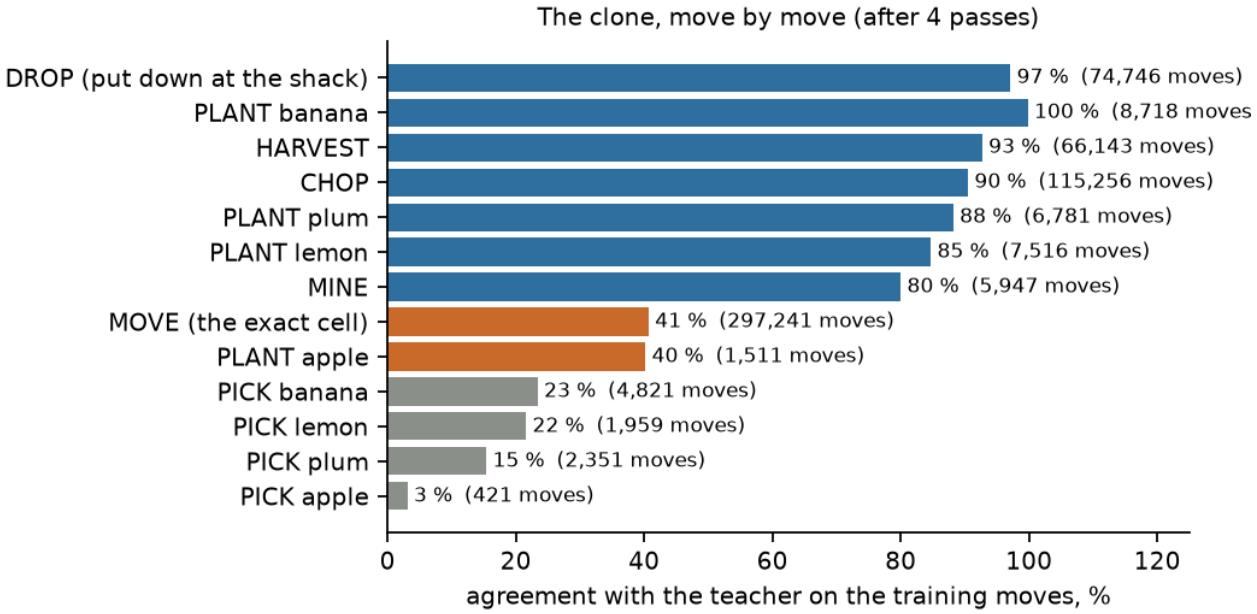

Figure 4: Agreement with the teacher, move by move. The routine moves are learned (drop, harvest, chop, plant); *move to exactly this cell* is the hard one (one cell of up to 242); *pick* (taking a fruit from the shack to plant it) is barely learned — it is rare and depends on a plan the clone does not have.

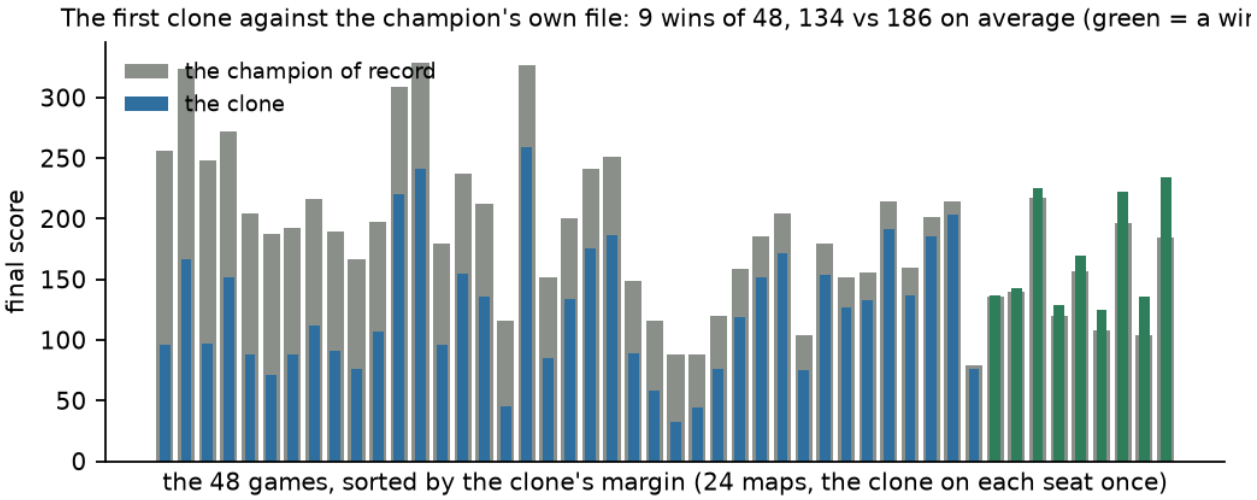

Figure 5: The bench: the clone against the champion's own submitted file, 48 games. Grey bars are the champion's scores, the narrow bars the clone's; green marks the clone's wins. A random-move policy scores 13.5 on this bench.

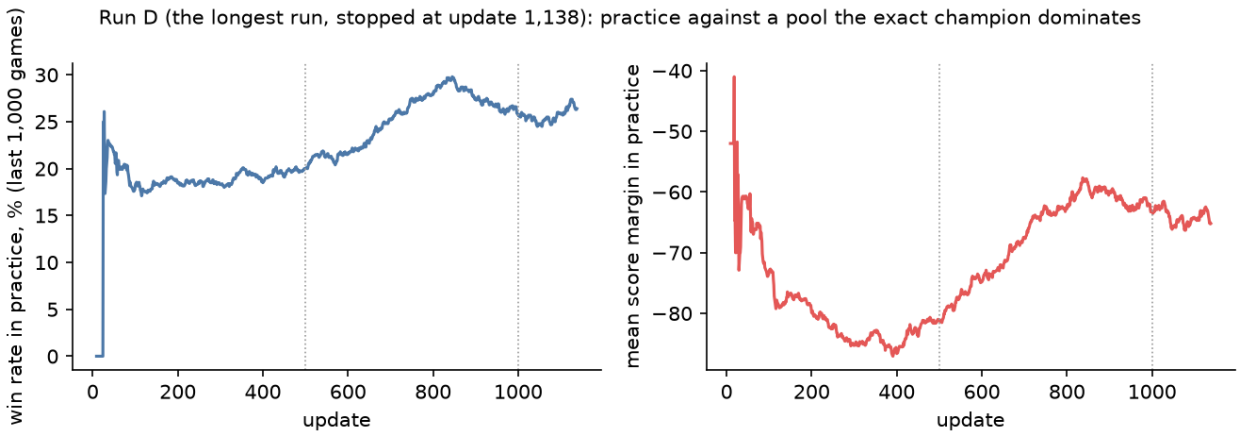

Figure 6: Run D from start to stop (1,138 updates): trained against a pool the exact champion dominates, so its practice win rate is a harder number than run A's was; it never rose. Both curves are over the last 1,000 finished games; the dotted lines mark the two snapshots tested in Figure 1.

3 2026-09-01 — the entropy gate, the credit path, the first lever

The overnight jobs, read. The three cluster jobs of 08-30 returned and were benched at their final snapshots against the champion's file: *a2* (full-parameter, 6,239 updates) **2 of 48**; *e2* (the corrected objective, 3,198) **3 of 48**; *i2* (the staged scope with the leash pinned, 5,414) **4 of 48**. The pinned leash slows the drift; it does not stop it. Every run ever benched has the same shape: the score peaks near the clone and decays with training. Nothing has passed 10 of 48; parity would need 24.

The second opinion, adopted. chatgpt_1's review of the whole experiment (08-31) corrected four claims of mine and named the causes in a new order: the trainer's credit horizon is the 32-step rollout buffer (about nine game turns), not the 300-turn game; run I's leash never actually faded; a 48-game bench is a ±5-win look, not a selector; and the *entropy bonus* — a term that rewards the planner for spreading its choice over 400 possible plans of which only 106 are ever seen from the teachers — was the best-fitting explanation of the staged drift. A recovery programme in numbered steps followed (`coordination/GOAL.md`): errata; a paired bench protocol (a 48-game scout, a locked 144-cell confirmation panel of 72 maps × 2 seats, effects read per cell); a measurement gate before any new run; then the entropy test under a gate frozen before its data existed.

Gate 0 — measure before running. Three instruments were built and reproduced: the gradient instrument (does the value estimate's learning bend the policy through the shared body? — for warmed-up runs, no: acquitted); the rollout telemetry (how many rows of a training buffer ever see a real game ending); and an independent calibration of the value estimate (it explains 4 % of the variance of the returns it predicts — nearly blind).

Step 4 — the entropy falsifier. Two arms from the same clone, same seed, same everything, differing in exactly one number (entropy bonus 0 versus 0.01), both on the cluster so that the machine cannot be confounded with the knob. The first attempt was preempted five times and its salvage kept only the newest checkpoint — twenty job-hours produced two snapshots at unrelated ages; the salvage was repaired to keep every checkpoint, and both arms were relaunched at the size the gate needs (2,709 updates) and, as insurance, on this computer as well. The cluster pair finished without preemption in 1.9 and 1.8 hours. Figure 7 shows the training side: the bonus does raise entropy (+0.068, interval $[0.051, 0.083]$ — the knob works) and changes nothing else (win rate +0.004 $[-0.004, +0.011]$; margin −0.02 $[-0.56, +0.52]$). Figure 8 shows the scouts at five ages: no age separates the arms, and both decay with depth. Figure 9 shows the gate itself: on the locked panel at updates 1,500 and 2,500 the bonus-off arm won **24 and 21** of 144, the bonus-on arm **23 and 22**; the paired effect of removing the bonus is **0.000 wins per cell, 95 % interval** $[-0.017, +0.021]$ (a bootstrap over the 144 map-seat units, both ages carried together, never pooled as 288 rows); the treatment arm loses no ground to the clone (net 0 cells of 6 allowed). **Verdict of record:** `ENTROPY_NOT_CONFIRMED`. The pair on this computer — same design, the full map corpus — gives the same verdict (18 / 20 against 23 / 22; effect −0.024 $[-0.056, +0.003]$). Two platforms, one answer: the bonus does not matter.

The credit path — the day's real finding. With the entropy suspect acquitted, the rollout telemetry was read for what the planner actually learns from. The trainer scores each decision by an estimate of the game's outcome; that estimate is a blend of *observed reward* (what really happened within its look-ahead) and the *value estimate's own opinion* of everything beyond. Measured on 50 million rows of the two arms: **observed reward supplies 2.3 % of the planner's learning signal; the value estimate supplies 97.7 %**, reproduced to two decimals by the second arm — against a value estimate that Gate 0 found nearly blind. (I first wrote that the reward was *absent*; that was wrong — the trainer's own trace delivers it within a turn — and the correction is on the card.) The reason is the reward's shape: in every run so far wood's whole value (4 points a piece) is paid in one lump on the final turn (`wood_shaping 0 + end_wood 4`), while the trainer looks ahead about nine turns of a 285-turn game. claude_1 then priced the two obvious levers offline on one set of games, with no training and no critic in the sentence (Figure 11): under the lump-sum reward the only rows in a training buffer carrying any observed reward are the 88 of 65,536 that are game endings — exactly, on three seeds; paying part of wood's value on delivery puts reward on 1,780 rows

(a factor of 20), and the environment's own default split (0.5 + 3.5) covers the same rows as 2 + 2 — only the size of the immediate signal differs; a 128-step rollout reaches a real ending 4.3 times as often (1.46 % → 6.21 % of rows). The two levers act on different rows, so neither replaces the other.

Step 5 — the first lever is training. Under the goal's standing authorization, the reward-path arm *r22* was launched on the cluster at 15:53Z: `wood_shaping 2 + end_wood 2` — wood still worth 4, half paid on the turn of delivery — everything else the bonus-on arm's recipe exactly (verified by argument diff and payload manifest), so that arm is the ready-made control and the same frozen gate reads the difference on the same 144 cells. It lands as this edition is written; its verdict goes on the card, the board and to chatgpt_1 the same way. After it, in the reviewer's order and one variable at a time: the longer rollout; whole-game returns for the planner; separating the value estimate's body from the policy's; target commitment.

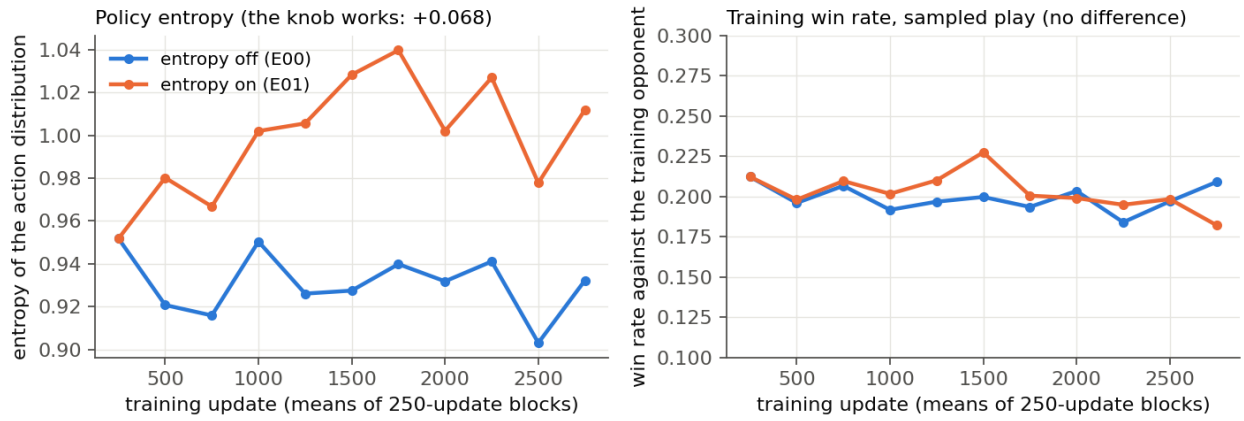

Figure 7: The training side of the entropy test, both cluster arms over their 2,709 updates (means of 250-update blocks). Left: the bonus does what it says — the entropy-on arm's action distribution is measurably wider throughout. Right: the training win rate (sampled play against the exact champion) does not care. This is evidence, not the verdict: the verdict is played out on fixed panels with the shipped decoding (Figure 9).

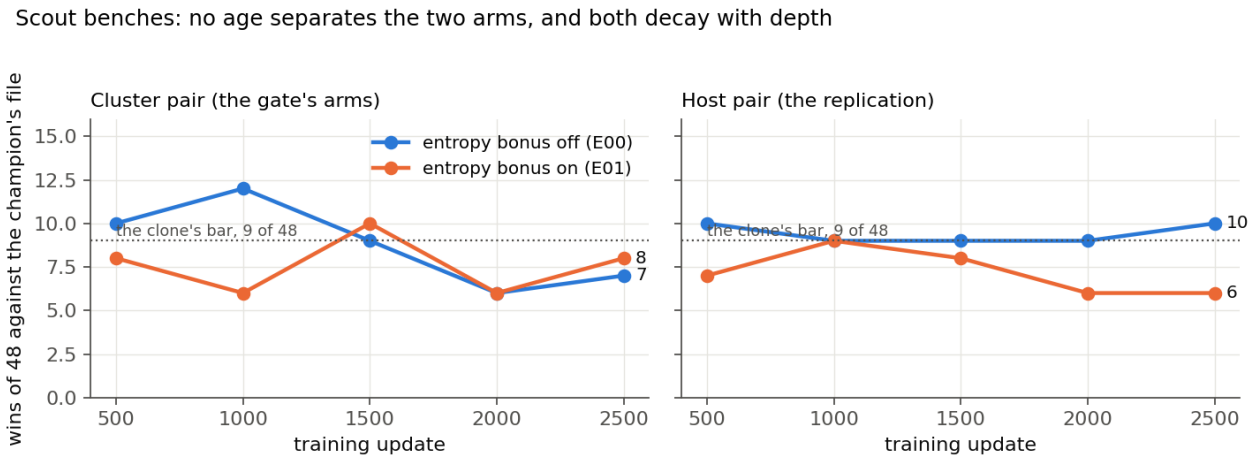

Figure 8: The scouts: each arm's checkpoints at five ages, 48 games against the champion's file, the same flags for every bench by construction. Left the cluster pair (the gate's arms), right the pair on this computer (the replication). A scout is a ±5-win look, not a verdict; what it shows is that no age separates the two arms, and that both decay with depth — the shape every run has shown.

The night's answers (09-01/02, added in the seventh edition). The lever was run and **the frozen gate confirmed it** — the programme's first positive result: *r22* (wood 2 + 2) won **31 and**

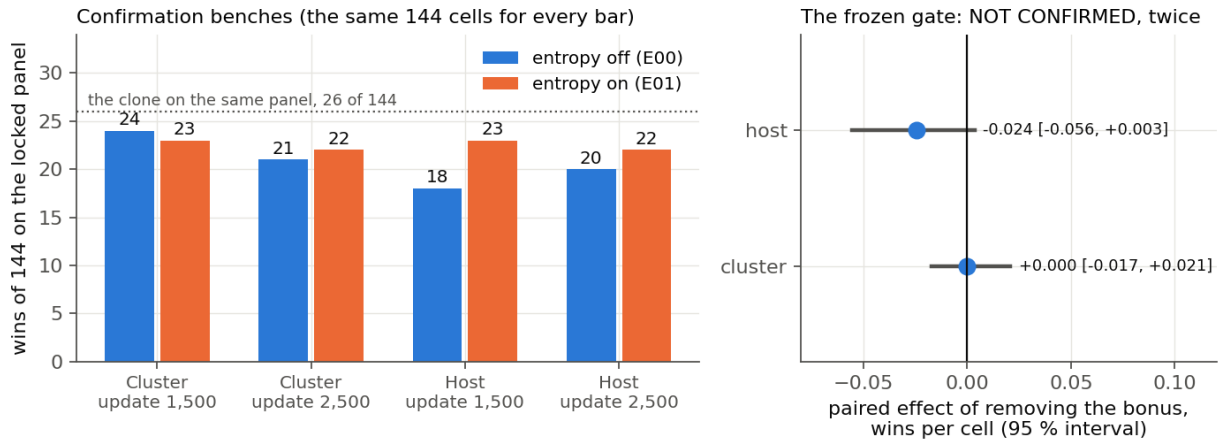

Figure 9: The frozen gate. Left: the confirmation benches on the locked 144-cell panel (72 maps × 2 seats, the same cells for every bar) at updates 1,500 and 2,500, cluster pair and host pair; the dotted line is the clone on the same panel. Right: the gate's number — the paired effect of removing the bonus, wins per cell, with its 95 % interval from a bootstrap over the 144 units. Both intervals contain zero: `NOT CONFIRMED`, twice.

29 of 144 on the locked panel against the control's 23 and 22 (paired effect +0.052 [+0.003, +0.101], positive at both ages, margin +8.3 points) and finished **net +11 cells above the clone — the first trained artefact to stand above its own starting point**. The host replication reproduced the effect's size (+0.049, interval touching zero exactly). Then the follow-ups, one variable each: the longer rollout *alone* is not confirmed (+0.017 [−0.004, +0.042]); the environment's default split 0.5 + 3.5 is no different from 2 + 2 (−0.017 [−0.066, +0.031]), so **2 + 2 stays the recipe**; and **the stack** — the split *plus* the 128-step rollout — posts the campaign's best numbers, **29 → 33 → 33 of 144** across updates 1,500 / 2,500 / 2,709, *rising with training age where every earlier run decayed* (its gain over the split alone is not yet separable from noise). Figure 10 is the whole campaign on one axis. The same stack with a doubled training budget runs on the cluster as this edition is written; its read is pre-registered as exploratory, and any promotion claim requires a fresh frozen gate written before the numbers are seen.

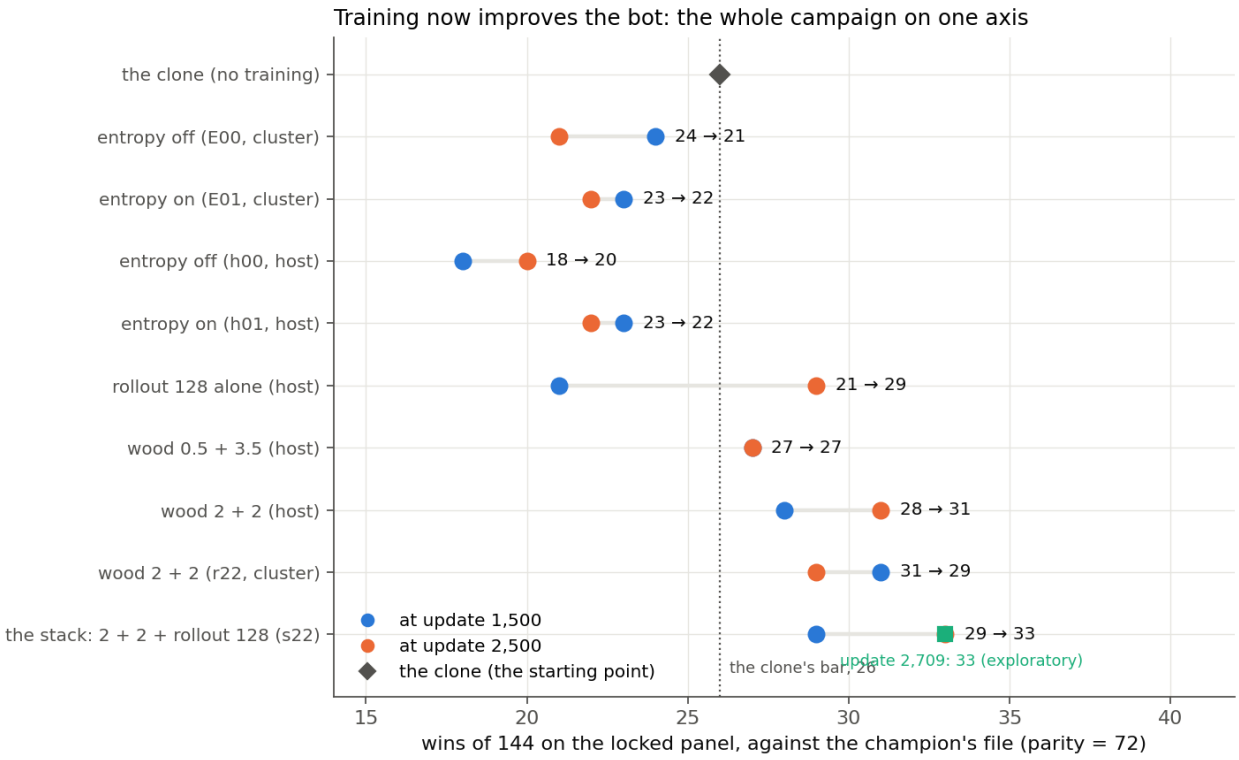

Figure 10: The campaign ledger: every arm of 2026-09-01/02 on the locked 144-cell panel against the champion's file, at training ages 1,500 (blue) and 2,500 (orange); the grey diamond is the clone every arm started from, the dotted line its 26 of 144; parity would be 72. The entropy arms sit at or below the clone; the wood split (2 + 2) lifts both platforms above it; the stack adds the long rollout and is the only arm whose score rises with age — 33 of 144 at its final checkpoint (green square, exploratory).

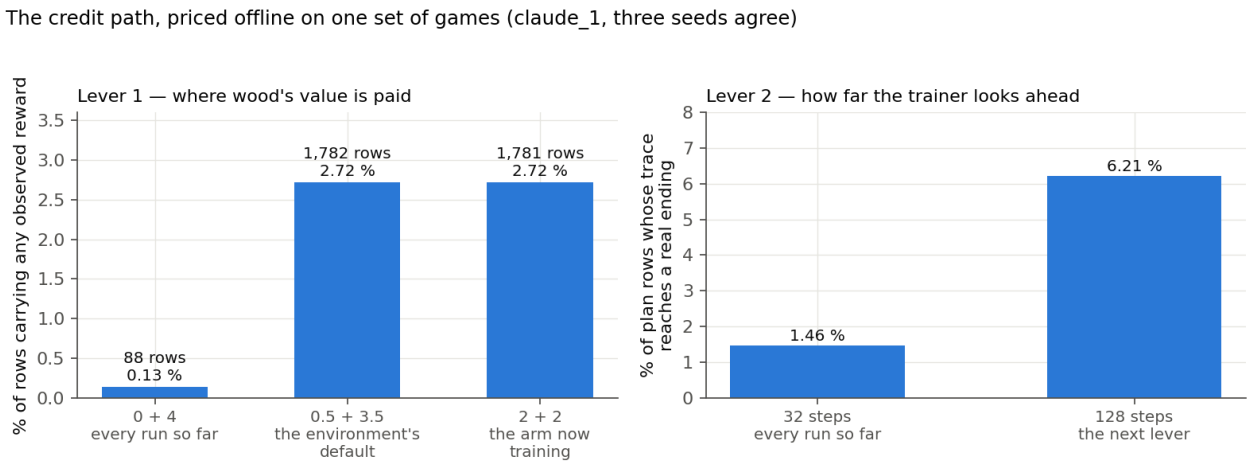

Figure 11: Why the runs decay, priced without a critic in the sentence (claude_1's instrument, one set of 65,536 training rows collected with the clone, replayed under each reward split and re-cut into each window; three seeds agree). Left: under the lump-sum reward used in every run so far, the only rows carrying any observed reward are the 88 that are game endings; paying part of wood's value on delivery puts reward on 1,780 rows — and the environment's own default split covers the same rows as the 2 + 2 now training. Right: a 128-step rollout lets the trainer's credit trace reach a real ending 4.3 times as often.

4 Where everything is

Two places: the repository (code, cards, reports — on `main`), and a data directory outside it (the big and the reproducible files).

The repository `/home/tarstars/prj/troll_farm` (the same tree as my worktree `.../troll_farm-local_claude_1`)

path	what it is
`coordination/BOARD.md`	the one page to read: what is in motion, your queue
`coordination/GOAL.md`	the target in full, and what I may and may not do without you
`coordination/tasks/20260829-nn-bot-way-b.md`	the card: the design, every ruling, the log — the record of this line
`coordination/tasks/20260829-nn-bot-way-b-env.md`, `...-dataset.md`	the two builders' contracts (Phases 1 and 2)
`local_claude_1/nn-bot/ANALYSIS-2026-08-29.md`	the analysis you read before deciding
`local_claude_1/nn-bot/OBS-PLANES.md`, `ENV-API.md`	the signed interface: the 104 things the network sees; the environment's calls
`rust/src/rl_full.rs`	the training environment (Rust): the game, 128 copies at once, the opponents
`cgauto/rl_full_env.py`	its Python side: `rewards, info = env.step(actions)`; the replay checker
`cgauto/train_level1_ppo.py`	the network itself (`SpatialActorCritic`) with its two heads
`local_claude_1/nn-bot/nn_runtime.py`	one shared adapter: the board to planes, the masks, the command codec, the end rule
`local_claude_1/nn-bot/build_dataset.py`	recorded games → the training set
`local_claude_1/nn-bot/train_clone.py`	the training set → the clone
`local_claude_1/nn-bot/train_ppo_full.py`	the clone → a better network by self-play
`local_claude_1/nn-bot/bench.py`	the judge: a network against a compiled bot on real maps; prints any game turn by turn
`local_claude_1/nn-bot/fake_full_env.py`	a stand-in environment for tests
`local_claude_1/nn-bot/results/clone-2026-08-30-a/`	**the clone, its 48 games, the reports, a README for your read**
`rust/src/strategies/champion_exact.rs`	the exact in-process copy of the champion (a training opponent), generated from the champion's own source; its proof in `codex_1/results/nn-bot-way-b-champion/`
`local_claude_1/nn-bot/yt_ppo_launcher.py`, `yt_ppo_entrypoint.py`	the cluster launcher (prepare / start / monitor / retrieve) and what runs inside a job — which now uploads a salvage copy of its newest checkpoint every half hour
`local_claude_1/nn-bot/grad_decompose.py`	the gradient instrument (claude_1): measures each learning term's push on every part of the network, and what one value-only step does to the decisions — the "why" of the collapse; one reviewer correction pending
`local_claude_1/nn-bot/export_full_actor.py`, `generate_full_bot.py`, `bed_full_bot.py`	Phase 4: checkpoint → integer weights; weights → the one Rust file; the file checked move for move against the Python clone, timed and counted
`cgauto/submissions/candidate-nn-clone.rs`	**the clone as one file** (52,854 characters; generated and functionally reproduced; not ladder-ready until the CPU fallback and the timing certification pass; not submitted); its readable form and the gate record in `codex_1/results/nn-bot-way-b-export/` (`REPORT.md` has every command and hash)
`coordination/tasks/20260829-nn-bot-way-b-champion.md`, `...-export.md`	the champion-opponent contract (done) and Phase 4's engineering contract (done on the host; claude_1's reproduction pending)
`local_claude_1/reconstructions/fits/reconstruct.py`	the exact rebuild of a recorded game, turn by turn
`tests/test_rl_full_env.py`, `tests/test_train_ppo_full.py`	the tests (7 and 42)
`docs/reports/2026-08-30-neural-network-line-progress.pdf`	this document

The data directory `/home/tarstars/nn-data/` (outside the repository)

path	what it is
`dataset-v400-2026-08-30/`	the training set: 817,811 decisions, 14 MB, with checksums
`clone-2026-08-30-a/`	the clone (`clone-pilot.pt`), its training log, both bench reports and the 48 games (also copied into the repository)
`ppo-2026-08-30-a/`, `-b/`, `-c/`	the three morning runs (their logs, snapshots and benches kept for diagnosis)
`ppo-2026-08-30-d/`	the longest run: `train.log` (one line per update), a snapshot every 250 updates, its two benches and their games (`bench-u500-replays.jsonl`, `bench-u1000-replays.jsonl`, readable with `bench.py -read`)
`ppo-2026-08-30-e/`, `-f/`	the two afternoon runs stopped from outside at 249 updates (logs only)
`ppo-2026-08-30-f2/`, `-g/`, `-h/`	the evening's three runs (the remedies on the mixed pool; champion-only; the undiscounted value target), each with its benches, games and log; `-f2/` also holds the decoding factorial's four bench files (`bench-clone-sampled*`, `bench-clone-ss*`)
`ppo-2026-08-30-i/`	**the staged run** (movement frozen): five benches with games, `bench-u500u2500`; its update-1,000 snapshot is the programme's best artefact (10 of 48)
`/home/tarstars/prj/troll_farm-local_claude_1/yt_work/ppo/`	the cluster payloads and retrieved outputs (staging, not in git)
`//home/delivery_ml/research/tarstars/troll_farm/runs/`	on the cluster (Cypress): one directory per job — inputs, outputs, logs
`/home/tarstars/nn-venv/`	the Python used for all of this (3.11, PyTorch 2.13 for CPU)
`/home/tarstars/prj/troll_farm/data/raw/games/`	the 24,973 recorded ladder games (7.1 GB), the raw material

5 The structure of the data

A recorded game (one file per game, 24,973 of them) holds the map at turn 0, both players' commands at every turn, and the referee's log. Positions and inventories are not stored per turn; `reconstruct.py` replays both players' commands through our copy of the rules and checks every step against the referee's log — on the 784 top-four games it disagrees with the referee on nothing.

The training set has three kinds of file in one directory:

- *states* (`states-pilot.jsonl.gz`, 12.9 MB): one compact line per turn — every troll's position, talents and cargo, every tree's kind, size, health and fruit, both banks — 58 bytes a turn, 224,400 turns;
- *maps* (`maps-pilot.json`): the 748 boards;
- *labels* (`labels-pilot.npz`, 1.3 MB): what the teacher did. Two kinds of row. A *plan row*, one per turn: which troll the player bought next (the number of that troll among 400 possibilities: speed 1–4, carry 1–5, harvest 0–3, chop 0–4; "nothing" when no purchase follows). A *command row*, one per troll per turn: the move, as one number among 13×242 — 13 kinds of action (move here, harvest, chop, drop, mine, plant four kinds, pick four kinds) over the 242 cells of the largest board; a *move* is labelled with the cell the troll actually reached.

The network's input is not stored: it is built when needed, by the same Rust code the environment uses, as **104 planes of 11×22 numbers** — the cell types, the trees, the trolls and what they carry, distances to the shacks and to the nearest trees, the banks, the scores, the troll being decided, the troll being saved for. (Stored, this would be 20 GB; built on the fly it costs 15 ms a row.)

Why 400 kinds of troll, not delineate's 144. His network chose among speed 1–3, carry 1–4, harvest 0–2, chop 0–3. A count over the top four's 1,725 purchases found 267 outside that box —

Bubaptik buys speed-4 trolls in half of its purchases. The game caps nothing, so the vocabulary follows the teachers.

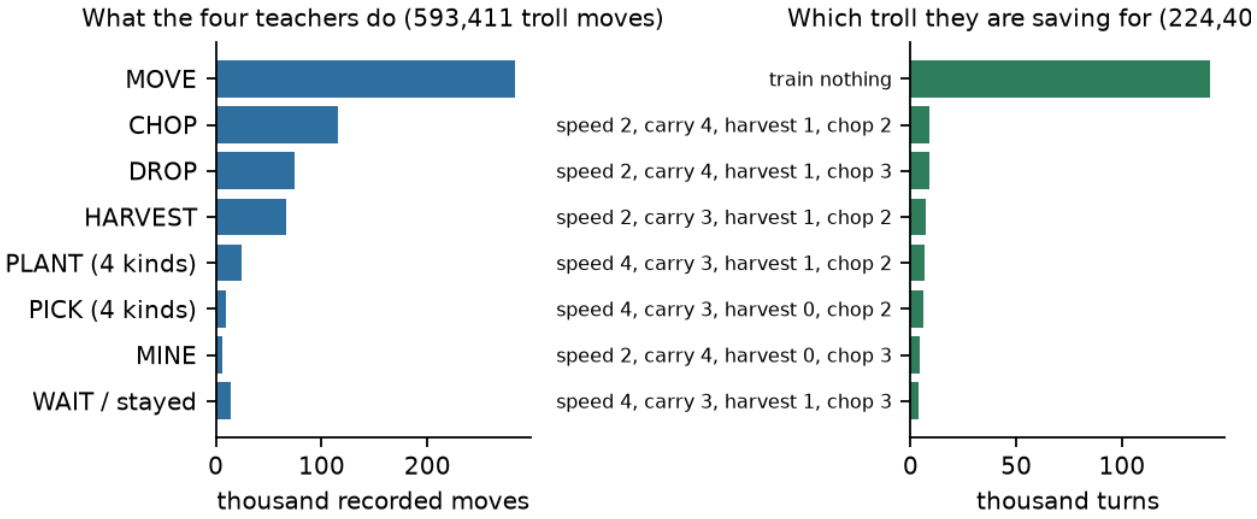

Figure 12: What the training set contains. Left: the moves of the four teachers' trolls. Right: which troll they were saving for, turn by turn ("nothing" after the last purchase of a game).

A checkpoint (`*.pt`) is four things: the network's weights, the optimizer's state, the settings it was trained with (including the vocabulary version `v400-2026-08-29` — a checkpoint from another vocabulary is refused), and the step count.

6 The structure of the programs

```
recorded games ---> reconstruct.py ---> build_dataset.py ---> training set
(24,973 files)      (exact turn-by-turn)  (labels + states)    (14 MB)
                                                                  |
                        rl_full.rs (the game, x128)               v
                        + rl_full_env.py  <----------------- train_clone.py
                              |                                   |
                              v                                   v
                       train_ppo_full.py  <--- anchor ---   clone-pilot.pt
                        (self-play, PPO)
                              |
                              v
                       snapshots (*.pt) ---> bench.py ---> games, wins, scores
                                                (vs the champion's file)
                              |
                              v   (Phase 4, built 08-30)
                       export_full_actor.py ---> integer weights
                                                      |
                                                      v
                       one Rust file  <--- generate_full_bot.py
                              |
                              v
                       bed_full_bot.py (move for move vs the Python
                       clone, timing, size) ---> the ladder, on the
                                                 owner's word
```

program	takes	gives
`reconstruct.py`	one recorded game	its exact state at every turn
`build_dataset.py`	a directory of recorded games	the training set (states, maps, labels); a report; a self-check of every label
`train_clone.py`	the training set, the Rust library	the clone checkpoint; a log per pass
`rl_full.rs` + `rl_full_env.py`	maps, opponents, actions	128 games stepping together; rewards; the planes and masks; a replay checker
`train_ppo_full.py`	a starting checkpoint, an anchor checkpoint, the environment	snapshots; one JSON line per update (win rate, margin, speed, how close to the anchor)
`bench.py`	a checkpoint, a compiled bot, a map slice	48 games with scores, purchases, ends; the games saved turn by turn
`nn_runtime.py`	(shared by the three above)	the planes, the masks, the command codec, the exact "can I train" check, the end-of-game rule
`export_full_actor.py`	a checkpoint	the integer weights (72,660 bytes) and a manifest; a verification checkpoint that plays exactly as the file will
`generate_full_bot.py`	the manifest and the weights	the one Rust file (and its readable form): the game, the planes, the network, the commands
`bed_full_bot.py`	the file, the clone's saved games, the champion's file	48 games compared command by command; the timing and size lines; the seat rule checked on 370 recorded games

The one decision every turn. A turn is several decisions in a row: first *which troll to save for* (one of 400, or nothing); then, for each own troll in order, *what it does* (one of 13×242, most of them masked as impossible); then the turn executes for both players. The reward is the final score difference, paid once per turn on the last decision.

7 The numbers so far

The environment's check	1,000 self-play games replayed through an independent rules copy: 0 disagreements, 0 illegal commands; 214 game-turns/s on the VM's 4 cores
The training set	817,811 decisions (224,400 plan + 593,411 command) from 748 games; 0 labels outside the vocabulary; built in 1 min 42 s
The clone, 4 passes	agreement with the teachers 74 % (which troll) and 65 % (what it does); Figure 4 per move
The clone on the bench	9 wins of 48 vs the champion's file (4 on seat 0, 5 on seat 1); 134 vs 186 points; 0 illegal commands, 0 timeouts; buys a troll at turn 1 in 44 games — the teachers' harvest-carrying trolls, not the champion's chopper
Self-play, run A (exploratory)	1,300 updates, 5.3 million decisions against the old practice mix; wins there 0 → 42 %; **its snapshot vs the champion's file: 2 wins of 48, 87 points to 183** — worse than the clone
Self-play, run C (exploratory)	the fixed trainer, the old mix, 250 updates: **3 wins of 48, 107 to 177** — the same lesson

Self-play, run D (the longest)	from the clone against the exact champion, 09:42Z–14:2xZ, 1,138 updates, 4.7 million decisions; **vs the champion's file: 3 wins of 48 (108 to 184) at update 500, 4 of 48 (82 to 169) at update 1,000**; a troll bought in 33, then 26 games (the clone: 44); practice win rate flat near 26 %
What the trolls do, per game	chop / harvest / plant / move: the clone 94 / 38 / 25 / 253; D at 500: 70 / 27 / 19 / 224; D at 1,000: 71 / 17 / 14 / 210 — less of everything; the purposeless pick-and-drop habit grew first (46 picks at update 500 against the clone's 33), then faded with the rest (Figure 2)
Self-play, runs E and F	the two remedies (the objective; the objective + warm-up), 249 updates each, stopped from outside at 14:41Z; F restarted as `ppo-f2` at 8 threads; E on the cluster
Self-play, run F2 (both remedies)	vs the champion's file: 5 of 48 (123.5 points) at update 500, **7 (132.2)** at 1,000, **2 (94.8)** at 1,500 — recovery, then the collapse
Self-play, runs G and H	G (champion-only): 5 (133.8) 4 (107.0); H (value target undiscounted): 3 (112.8) 8 (132.9) 2 (109.3) — the same shape; H's value estimate fit collapsed (explained variance 0.25)
The decoding factorial (the clone)	plan Œ command, most-likely / drawn: **9 (133.9) / 8 (133.5) / 3 (109.2) / 4 (103.4)** — the command decoding carries the whole gap; drawn play wanders (329 moves a game vs 253)
Run I (staged: movement frozen)	vs the champion's file: **9 (128.6) 10 (131.0) 9 (127.7) 6 (124.1) 5 (121.5)**; the only run that neither eroded nor collapsed; purchases 43–44 of 48 throughout; its practice numbers tracked the bench for the first time (17.7–20.9 %)
Phase 4: the clone as one file	52,854 characters; 48/48 games and 13,206/13,206 commands identical to the Python clone (both seats); first turn 14.8 ms, ordinary turns 6.5 ms median, 10.6 ms at the 99th percentile; the seat rule true on 370/370 recorded games; codex_1's numbers and the host's agree
The exact champion opponent	matches the champion's submitted file on 200/200 games and 49,945/49,945 turns, raw and gameplay commands; reproduced; 1,058 turns/s
The cluster	the smoke job: 899 decisions/s on 16 cores; the first six jobs died at their wall-clock limits (my inconsistent budget) or were aborted — nothing retrievable; rebuilt with half-hourly salvage uploads; **three jobs at ∼1,040 decisions/s, 60 million each**: a2 (full-parameter, long horizon), e2 (corrected objective), i2 (staged, the leash pinned at 0.1); results ∼19:00–20:00Z
This computer	20 cores, no GPU: the network answers 17,000 boards/s in batches; the bottleneck is building the planes and running the opponents
The overnight cluster jobs (08-31)	a2 (full-parameter) at update 6,239: **2 of 48**; e2 (corrected objective) at 3,198: **3 of 48**; i2 (staged, leash pinned) at 5,414: **4 of 48** — depth hurts; the pinned leash slows the drift, does not stop it

Gate 0 (09-01)	the value estimate's learning acquitted for warmed-up runs; the value estimate explains 4 % of its targets' variance; the credit trace reaches a real ending on 1.8 % of rows
The entropy gate (09-01)	cluster pair, 2,709 updates each, one field differs: locked panel **E00 24 / 21, E01 23 / 22 of 144** at updates 1,500 / 2,500; paired effect **0.000** $[-0.017, +0.021]$; clone non-inferiority net 0; `ENTROPY_NOT_CONFIRMED`; the host pair replicates it (18 / 20 vs 23 / 22; -0.024 $[-0.056, +0.003]$); scouts E00 10 / 12 / 9 / 6 / 7, E01 8 / 6 / 10 / 6 / 8; training side: entropy $+0.068$, win rate and margin null
The credit path (09-01)	the planner's learning signal is **2.3 % observed reward, 97.7 % the value estimate**, reproduced by the second arm; under the lump-sum reward, reward rows = game endings exactly (88 / 82 / 84 of 65,536 on three seeds); shaping on: ∼1,780 rows (×20); 128-step rollout: 4.3× the traces reaching an ending
The first lever (09-01)	*r22*: wood 2 + 2, otherwise the bonus-on arm's recipe; launched 15:53Z on the cluster; verdict by the same frozen gate against the bonus-on arm as control

8 How to read the clone's games

```
cd /home/tarstars/prj/troll_farm
PYTHONPATH=. /home/tarstars/nn-venv/bin/python local_claude_1/nn-bot/bench.py \
    --read local_claude_1/nn-bot/results/clone-2026-08-30-a/\
bench-argmax-replays.jsonl --game 10
```

`-game` is 1 to 48 (0 prints all). Each line is one turn: the champion's commands on the left, the clone's on the right. The table of games (map, seat, scores, purchases, how it ended) is `bench-argmax.json`. Game 10 is a win (225 to 217); game 1 a loss with no purchase (76 to 167); game 34 has the clone's one long loop. The same command reads any run's games — the programme's best snapshot's are `/home/tarstars/nn-data/ppo-2026-08-30-i/bench-u1000-replays.jsonl`.

What I saw reading them: the second troll works like a teacher's — walks to trees, harvests, plants lemons, brings fruit home; the first troll often *picks a fruit from the shack and drops it back*, for stretches of ten or twenty turns — copying without a purpose (pick and drop are 13 % of the recorded moves). It does not chop as a plan. Those two habits are what self-play must unlearn first.

9 Risks and what would stop the line

- *Self-play finds a hole in our copy of the game rather than a strategy.* Guard: samples of its games are replayed through the independent rules copy at every check.
- *What the network trains against decides what it learns* — observed today, not feared: three runs against our weak practice bots got worse against the champion. The runs now train against an exact copy of the champion; the bench uses the champion's real file; the target adds orchard 6; the ladder hour is the truth. The next opponent to add is orchard 6 itself.
- *Self-play has not yet beaten the clone, and full-parameter self-play reliably destroys it* — the day's mapped result, not a fear. What is known: the staged freeze stops the destruction; the leash must

not fade; the value estimate's learning path through the shared network body is the prime suspect and is being measured. What answers next: the three overnight cluster jobs and the gradient instrument. The card's budget for Phase 3 (2–4 weeks) holds — two days are spent.

- *The cluster eats what it is told to eat.* Two lessons paid for in full: a job's step budget and its wall-clock limit must be set consistently (thirteen hours of checkpoints died inside two jobs for my inconsistent pair), and anything that lives only inside a job is lost when the job is — every job now uploads a salvage copy of its newest checkpoint every half hour.
- *Speed on the platform.* Measured now, not estimated: the one-file clone plays a turn in 6.5 ms (10.6 ms at the 99th percentile) on this computer against a limit of 50; the platform's machines are slower, so the file keeps a margin of several times.
- *Dead conditions on the card:* no gain over the clone after 200 million decisions; or a hole in the engine — then the run stops and the card says so.

The agents and the routine

codex_1 and *claude_1* build and reproduce each other's work on the VM; *chatgpt_1* audits on your instruction; I design, rule, integrate onto `main`, train on this computer and write to you. Every ruling and every number is in the card's log; the board's queue item 0 is the milestone note. No platform action has been taken since your word on 08-29 and none will be before Phase 4 and your prediction.